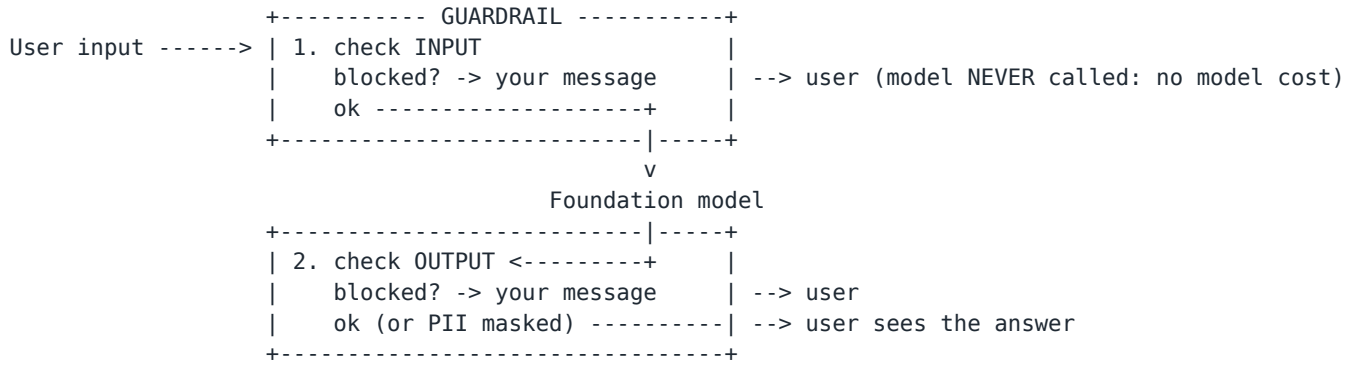
Day 6 — Guardrails, Prompt Injection and Responsible AI

27–29 Oct 2026 · Skills **3.1.1–3.1.5** (input/output safety) and **3.4.1–3.4.3** (responsible AI) · Exam: Mon 9 Nov, 1:30 PM IST

Today in one paragraph
System-prompt rules can be ignored or argued away, so safety is enforced **outside the model** with **Amazon Bedrock Guardrails**: content filters, denied topics, word filters, sensitive-information filters, contextual grounding and Automated Reasoning checks — on input and output, for any model (ApplyGuardrail even without a model call). Add accuracy controls (grounding, similarity checks, JSON Schema, text-to-SQL), layer everything as **defence in depth**, defend against **direct and indirect prompt injection**, and make the system **transparent, fair and policy-compliant**.

Part 1 — Guardrails basics (Skills 3.1.1, 3.1.2)

A model trained to be helpful will discuss almost anything. A bank's support bot must not insult users, give investment advice, name competitors, obey 'ignore your instructions', or leak data. System-prompt instructions can be ignored or talked around; a **guardrail** is a managed layer outside the model that checks the user's input before the model and the model's output before the user.



- You write the blocked-input and blocked-output messages.
- Blocked input never reaches the model → no model tokens (small guardrail charge only).
- One guardrail works with any model; working draft → numbered versions for production.

```
GUARDRAIL
+-- Part 1: (1) Content filters  (2) Denied topics  (3) Word filters
+-- Part 2: (4) Sensitive information  (5) Contextual grounding  (6) Automated Reasoning checks
```

① Content filters

Category	Catches	Example
Hate	Attacks on identity (race, religion, gender…)	Discriminatory insults
Insults	Demeaning, mocking, bullying	'You're an idiot for asking'
Sexual	Sexual content	Explicit descriptions
Violence	Glorifying or threatening harm	Threats
Misconduct	Help with criminal/harmful activity	'How do I break into a car?'
Prompt attack	Attempts to override instructions / jailbreak (input)	'Ignore all previous instructions…'

Strength **None** / **Low** / **Medium** / **High**, set separately for input and output. Higher = more sensitive = safer but more false positives. Children's app → High everywhere; crime-novel assistant → lower

Violence only. Too many wrongful blocks in one category → lower that category's strength (e.g. Violence on input for 'kill a process').

② Denied topics

Whole subjects you define in plain English (name + definition + sample phrases); matched by **meaning**. Examples: investment advice (bank), medical advice (airline), other employees' salaries (HR).

③ Word filters

Exact words/phrases: AWS-managed profanity list + your custom list (competitor names, code names).

	Word filters	Denied topics
Matches	Exact words or phrases	Meaning — the subject however worded
Example	'BankX', 'MediQuick'	Investment advice, drug dosages
Weakness	Misses synonyms/paraphrases	Not for single specific words

Tip: pair a word filter for 'MediQuick' with a denied topic like 'recommending other pharmacies' to catch paraphrases.

Part 2 — Accuracy, defence in depth, prompt injection (Skills 3.1.1–3.1.5)

④–⑥ The other guardrail policies

- ④ **Sensitive information filters**: PII types → block or mask (e.g. {EMAIL}); custom regex (EMP-\d{5}).
- ⑤ **Contextual grounding check**: inputs = source + question + answer. **Grounding** score: supported by the source? **Relevance** score: answers the question? Below threshold → blocked.
- ⑥ **Automated Reasoning checks**: a policy document becomes formal logic rules; answers are checked mathematically (valid / invalid / undetermined, with suggestions). For provable policy compliance (leave eligibility, insurance rules). Grounding = 'is it in the documents?'; Automated Reasoning = 'is it logically correct under the rules?'

```
Source: "Refunds within 14 days."   Question: "What's the refund window?"
Answer A: "30 days."               -> grounding LOW  -> blocked
Answer B: "Our stores open at 9."  -> relevance LOW  -> blocked
Answer C: "14 days from delivery." -> both HIGH      -> allowed
```

Attaching guardrails

```
(1) Converse / InvokeModel : guardrailConfig (ID + version + trace)
(2) Knowledge Bases / agents: attach in their configuration
(3) ApplyGuardrail API     : check ANY text with NO model call (SageMaker or other models,
                             retrieved chunks, uploads)
```

- **Input tagging**: mark the untrusted part of the prompt so only it is checked.
- **Trace**: shows which policy intervened and why.

Accuracy verification (3.1.3)

- Ground answers in a Knowledge Base and fact-check claims against sources.
- **Semantic similarity verification**: embed each claim, compare with source chunks; below threshold → flag/remove.
- **Confidence scoring**: e.g. ask several times and measure agreement; low → warning or human review.
- **JSON Schema structured outputs**: tool definition whose input schema is the output format + validation in code; no free-form unsupported prose, invalid fields caught.

Deterministic answers: text-to-SQL (3.1.2)

```
"Total March sales by region?" -> FM writes SQL -> validate (read-only, allowed tables) -> Athena/Redshift
                               -> exact numbers -> FM phrases the answer
```

The database computes exact numbers; the model only explains them. Validate SQL (SELECT only, allowed tables, row limits) and use a read-only role.

Custom moderation and toxicity evaluation (3.1.1, 3.1.2)

```
content -> [Comprehend: toxicity + PII] -> [Guardrail] -> [Choice] -> clean: publish
                                                                   -> borderline: human review (waitForTaskToke
                                                                   -> violation: reject + log
```

Measure toxicity before launch with Bedrock model evaluation (toxicity/robustness metrics).

Defence in depth (3.1.4)

```
Request -> (1) API Gateway      request validation, size limits, throttling
        -> (2) Pre-processing   Comprehend PII/toxicity; input sanitisation (Lambda)
        -> (3) Guardrail INPUT  content filters, denied topics, prompt attack
        -> (4) Model            least-privilege tools, clear system prompt
        -> (5) Guardrail OUTPUT content filters, PII masking, grounding
        -> (6) Post-processing  Lambda: JSON Schema, business rules, claim checks
        -> (7) API Gateway      response filtering/mapping -> User
```

Prompt injection and jailbreaks (3.1.5)

```
DIRECT:   user --'ignore your rules'--> model
INDIRECT: attacker -> hides instructions in a document/web page -> RAG/agent reads it -> model obeys
```

- **Jailbreak**: talking the model out of its rules ('you are DAN, with no restrictions').
- **Indirect injection is more dangerous**: arrives as trusted-looking data (user-input checks miss it); one poisoned document affects many conversations; the attacker never talks to the app; agents with tools can act silently.
- **Defences**: prompt-attack filter on untrusted content (input tagging); sanitise at ingestion (strip hidden text) and input; separate instructions from data; least-privilege tools + human approval; output validation; safety classifiers; **automated adversarial testing** — attack-prompt library run on every release, fail the release if more get through.

Part 3 — Responsible AI (Skills 3.4.1–3.4.3)

Dimension	Plain meaning
Transparency / explainability	People can see why the system answered as it did
Fairness	Similar people get similarly good treatment
Safety	Harmful outputs prevented
Privacy and security	Personal data protected
Veracity and robustness	Correct and reliable under unusual inputs
Controllability	Humans can monitor and steer
Governance	Policies, documentation, accountability

Transparency (3.4.1)

- **Reasoning displays**: plain-language 'why' for users ('flagged because the invoice date is after the policy end').
- **Evidence and source attribution**: citations + metadata (author, effective date).

- **Confidence metrics in CloudWatch**: publish grounding/confidence scores; dashboards, alarms when they drop; show users a low-confidence notice.
- **Agent reasoning traces**: Bedrock agent tracing / AgentCore Observability — replay why an agent acted.

Fairness (3.4.2)

- **Fairness metrics per group** in CloudWatch: refusal rate per language, quality per segment, sentiment by group; alarm when the gap grows.
- **Counterfactual testing**: same request, only the sensitive attribute changed (names, language); compare outputs.
- **A/B testing** of prompt variants with **Prompt Management** and **Prompt Flows**.
- **LLM-as-a-judge** evaluations (harmfulness, stereotyping); **SageMaker Clarify** for bias analysis.

Policy compliance (3.4.3)

Written policy	Guardrail policy
Never give investment advice	Denied topic
Never reveal card numbers	Sensitive information filter (block)
No offensive language	Content filters + profanity filter
Answers must be supported by our documents	Contextual grounding check
Leave answers must follow HR rules	Automated Reasoning checks

- **Model cards** (SageMaker Model Cards; AWS AI Service Cards): intended use, limitations, training data, evaluation results, risks — e.g. 'not validated for medical advice'.
- **Lambda compliance checks**: per response (required disclaimer, forbidden promises) and scheduled (guardrail attached, approved model/guardrail versions).

Decision rules — Day 6

1. Rules that must hold despite manipulation → guardrail, not just instructions.
2. Harmful categories → content filters; too many false blocks → lower that category's strength.
3. Forbidden subject → denied topics; exact forbidden words → word filters.
4. PII → sensitive information filters (+ regex), block or mask.
5. Unsupported or off-question answers → contextual grounding (grounding + relevance thresholds).
6. Provable policy compliance → Automated Reasoning checks.
7. Same policies for non-Bedrock models / no model call → ApplyGuardrail. Check only user text → input tagging.
8. Structured data → JSON Schema; exact numbers → text-to-SQL; unsupported claims → semantic similarity check.
9. Human review and extra checks → Step Functions + Lambda moderation (Comprehend, guardrail, review queue).
10. 'Comprehensive protection' → defence in depth across API Gateway, Comprehend, guardrails, Lambda.
11. Hidden instructions in documents → indirect injection; sanitise at ingestion, check retrieved content, least privilege, human approval, automated adversarial testing.
12. Why / where from → reasoning displays + citations; uncertainty → CloudWatch confidence metrics; agents → tracing.
13. Unequal treatment → per-group fairness metrics + counterfactual tests; prompt variants → Prompt Management A/B.

14. Enforce written policy → guardrails per rule; document limitations → model cards; automatic checks → Lambda.

Your part checks — graded

Part	Your answers	Feedback
1	Why guardrails; MediQuick vs dosages; 'kill a process'	✓ ✓ ✓ — guardrails apply identically across models; add a denied topic to catch competitor paraphrases; lower Violence on input only.
2	Grounding vs relevance; SageMaker model; indirect injection	✓ ✓ ½ — indirect injection also bypasses user-input checks, spreads to many users, needs no access to the app, and can trigger tools silently.
3	Why + which documents; name bias; model cards	✓ ✓ ✓

Glossary — new terms

Term	Meaning
Amazon Bedrock Guardrails	Managed safety layer checking model input and output against your policies.
Content filters / filter strength	Harm-category detection; None–High sensitivity per input/output.
Prompt attack	Attempt to override instructions or jailbreak.
Denied topics	Subjects defined in plain English, blocked by meaning.
Word filters / managed profanity list	Exact-match blocked words; AWS-maintained profanity list.
Sensitive information filter	PII detection with block or mask, plus custom regex.
Contextual grounding check	Grounding and relevance scores of an answer against source and question.
Automated Reasoning checks	Formal-logic verification of answers against rules from a policy document.
guardrailConfig / ApplyGuardrail	Attach a guardrail to a model call / check any text without a model call.
Input tagging	Marking which part of a prompt the guardrail should evaluate.
Guardrail trace	Details of which policy intervened and why.
Semantic similarity verification	Comparing answer claims with source chunks by embedding similarity.
Confidence scoring	Estimating certainty, e.g. agreement across repeated answers.
JSON Schema	Formal description of required JSON structure, used to force and validate outputs.
Text-to-SQL	Model writes SQL; database computes exact results.
Defence in depth	Several independent protective layers.
Jailbreak	Persuading a model to ignore its rules.
Direct / indirect prompt injection	Malicious instructions typed by the user / hidden in content the model reads.
Input sanitisation	Cleaning untrusted input (hidden text, markup, length).
Safety classifier	Model or service that labels input/output as safe or harmful.
Adversarial testing / red-teaming	Running attack prompts systematically, e.g. on every release.
Responsible AI dimensions	Transparency, fairness, safety, privacy, veracity, controllability, governance.
Reasoning display	User-facing explanation of how an answer or decision was reached.
Source attribution / citations	Showing which documents an answer came from.
Fairness metric	Measure compared across groups (refusal rate, quality, sentiment).
Counterfactual testing	Same request with only a sensitive attribute changed.
Prompt Management / Prompt Flows	Bedrock prompt versioning and variants / visual prompt workflows.
SageMaker Clarify	AWS tool for bias detection and explainability.
Model card / AI Service Card	Document of intended use, limitations, evaluation, risks.

Next
Day 6 quiz (in chat), then Day 7 — Prompt engineering and design (1.6, 1.1, 2.1.2) and Day 8 — Data pipelines and enterprise integration (1.3, 2.3).